Note to myself

- Clone repo:

```bash
git clone https://github.com/ymirsky/Kitsune-py.git
cd Kitsune-py
pip install scapy numpy scipy
```

- The repo downloaded already have the dataset we need

Option 2: Get it directly from the Kitsune GitHub repo

If you just want the Mirai `.pcap` to test the Kitsune Python code you cloned earlier, the author actually hosts a small, dedicated ZIP of the Mirai PCAP right inside their GitHub repo for easy testing.

You can download it directly to your machine without needing the Kaggle API at all:

```
wget https://github.com/ymirsky/Kitsune-py/raw/master/mirai.zip
unzip mirai.zip
```

This will instantly give you the `mirai.pcap` file ready to feed into the Python script!

- Install cython to fix bug from the code: pip install cython
- Install tshark and then turnoff AppArmor for it: “sudo apt install tshark -y”

```bash
sudo apt-get update
sudo apt-get install apparmor-utils
sudo aa-disable /usr/bin/tshark
```

- Run this to fix the numpy legacy error: “ find ~/final/Kitsune-py -type f -name "*.py" -exec sed -i 's/numpy\.Inf/numpy.inf/g' {} +”
- Fix the code first

How to fix it:

Instead of pickling the entire `Kitsune` object, you only want to pickle the `KitNET` neural network (which holds all the actual weights and anomaly detection logic).

Change the saving block to:

```python
# 5. Disk Persistence
print("Writing model weights to persistent disk...")
with open("kitsune_mirai_model.pkl", "wb") as f:
    # Save just the neural network, not the packet parser
    pickle.dump(K.AnomDetector, f)
print("Model successfully saved as 'kitsune_mirai_model.pkl'.")
```

2. Tshark vs. Scapy (Speed bottleneck)

In your parameters, you set `packet_limit = np.Inf`. The Mirai pcap is usually hundreds of thousands of packets.

Kitsune will automatically look for `tshark` (Wireshark's command line tool) on your system. If it doesn't find it, it falls back to `scapy`.

- **If it uses Tshark:** It runs relatively fast.
- **If it uses Scapy:** It is *excruciatingly* slow. A 500,000 packet pcap could take 10+ hours.

Recommendation: Make sure `tshark` is installed on your working VM before running this script:

```
sudo apt install tshark -y
```

FULL code here: (remember to fix it first)

```
from Kitsune import *
import numpy as np
import time
import pickle

# 1. Dataset and Run Parameters
path = "mirai.pcap"
packet_limit = np.Inf

# 2. KitNET Architecture Parameters
maxAE = 10         # Maximum size for any autoencoder in the ensemble layer
FMgrace = 5000     # Number of packets used to incrementally learn the feature mapping
ADgrace = 50000    # Number of packets used to train the anomaly detector (the ensemble)

# 3. Pipeline Initialization
print(f"Loading {path} and initializing the Kitsune pipeline...")
K = Kitsune(path, packet_limit, maxAE, FMgrace, ADgrace)

print("Starting packet processing and model training...")
RMSEs = []
i = 0
start_time = time.time()

# 4. Training Loop (Online Learning)
while True:
    i += 1

    # Terminal feedback to ensure the SSH session does not appear frozen
    if i % 25000 == 0:
        print(f"Processed {i} packets...")

    # Extract features, update the model, and score the packet
    rmse = K.proc_next_packet()

    # -1 indicates end of file or parsing error
    if rmse == -1:
        break

    RMSEs.append(rmse)

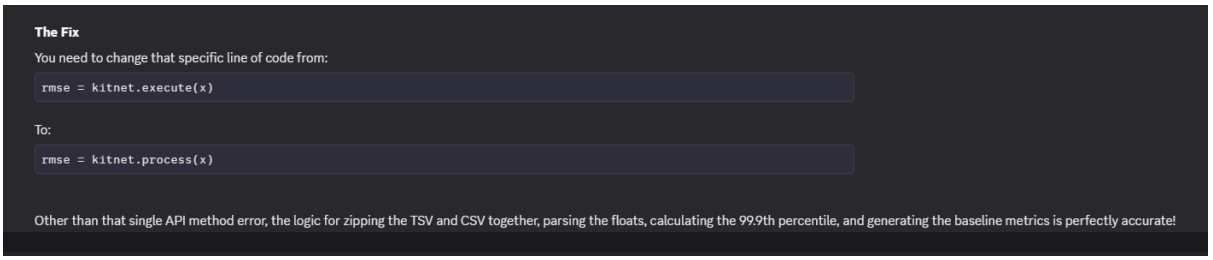
end_time = time.time()
print(f"\nPipeline complete. Total packets processed: {i-1}")
print(f"Execution time: {end_time - start_time:.2f} seconds")

# 5. Disk Persistence
print("Writing model weights to persistent disk...")
```

```
with open("kitsune_mirai_model.pkl", "wb") as f:
    pickle.dump(K, f)
print("Model successfully saved as 'kitsune_mirai_model.pkl'.")
```

Part 2:

(evaluate_threshold.py)




```
import pickle
import numpy as np
import csv
import time

model_path = "kitsune_mirai_model.pkl"
tsv_path = "mirai.pcap.tsv"
csv_path = "mirai_labels.csv"

# 1. Load the trained KitNET ensemble
print(f"Loading defensive model from {model_path}...")
with open(model_path, "rb") as f:
    kitnet = pickle.load(f)

benign_rmses = []
malicious_rmses = []

print("Starting inference across feature vectors...")
start_time = time.time()

# 2. Stream through both files simultaneously
with open(tsv_path, "r") as f_tsv, open(csv_path, "r") as f_csv:
    tsv_reader = csv.reader(f_tsv, delimiter='\t')
    csv_reader = csv.reader(f_csv)

    # Skip the single header row in the TSV discovered in your consistency check
    next(tsv_reader)

    i = 0
```

```
    for tsv_row, csv_row in zip(tsv_reader, csv_reader):
        i += 1

        # Parse the 100-feature vector and the label
        x = np.array(tsv_row, dtype=float)
        label = int(csv_row[0])

        # 3. Pure Inference: use execute() to score the vector without updating weights
        rmse = kitnet.process(x)

        # 4. Categorize the score based on the true label
        if label == 0:
            benign_rmses.append(rmse)
        else:
            malicious_rmses.append(rmse)

        if i % 100000 == 0:
            print(f"Scored {i} samples...")

end_time = time.time()
print(f"\nInference complete in {end_time - start_time:.2f} seconds.")

# 5. Calculate the Threshold and Baseline Metrics
benign_rmses = np.array(benign_rmses)
malicious_rmses = np.array(malicious_rmses)

# Set the detection threshold at the 99.9th percentile of benign traffic
# This accounts for natural network noise while remaining strict
threshold = np.percentile(benign_rmses, 99.9)

detected_attacks = np.sum(malicious_rmses > threshold)
missed_attacks = np.sum(malicious_rmses <= threshold)
detection_rate = (detected_attacks / len(malicious_rmses)) * 100
evasion_rate = 100 - detection_rate

print("\n--- Baseline Defensive Metrics ---")
print(f"Total Benign Packets: {len(benign_rmses)}")
print(f"Total Malicious Packets: {len(malicious_rmses)}")
print(f"Calculated Alert Threshold: {threshold:.4f}")
print(f"Baseline Detection Rate: {detection_rate:.2f}%")
print(f"Baseline Evasion Rate: {evasion_rate:.2f}%")
```